Universidad El Bosque

Bases de Datos 2

Solución guiada - Quiz 2
SQL Avanzado en Oracle 19c

Joins · Subconsultas · CTE · Funciones analíticas · Jerarquías · Top-N

Documento de revisión para clase

1. Objetivo de la revisión

Este documento desarrolla paso a paso las dos preguntas del Quiz 2. La idea no es memorizar una consulta final, sino entender cómo separar el problema en capas, identificar errores de granularidad y joins, y decidir en qué momento aplicar agregaciones y funciones analíticas.

2. Esquema de trabajo

```
CLIENTE (id_cliente PK, nombre, ciudad, segmento, fecha_registro)
SUCURSAL (id_sucursal PK, nombre, ciudad, region)
CATEGORIA (id_categoria PK, nombre, id_categoria_padre FK -> CATEGORIA)
PRODUCTO (id_producto PK, nombre, id_categoria FK, precio_lista)
PEDIDO (id_pedido PK, id_cliente FK, id_sucursal FK, fecha_pedido, estado)
DETALLE_PEDIDO (id_pedido FK, num_linea, id_producto FK, cantidad,
                precio_unitario, descuento)
PK (id_pedido, num_linea)
DEVOLUCION (id_devolucion PK, id_pedido, num_linea, fecha_devolucion,
            cantidad_devuelta, motivo)
FK (id_pedido, num_linea) -> DETALLE_PEDIDO
```

Regla de negocio: descuento se expresa entre 0 y 1. El valor de una línea es cantidad × precio_unitario × (1 - descuento).

3. Pregunta 1 - Diagnóstico y reescritura

Requerimiento: para cada cliente PREMIUM, listar sus pedidos de 2025 con el valor total, el acumulado de compra hasta ese pedido, el número de devoluciones no asociadas a GARANTIA y únicamente los tres pedidos de mayor valor de cada cliente.

3.1 Cómo analizar la consulta antes de corregirla

- Primero determine la granularidad deseada: una fila por pedido.
- Después identifique qué tablas pueden multiplicar filas: DETALLE_PEDIDO y DEVOLUCION.
- Luego separe agregaciones por nivel: total del pedido por un lado y conteo de devoluciones por otro.
- Finalmente aplique funciones analíticas sobre el conjunto ya agregado.

3.2 Defecto 1 - Función analítica dentro del WHERE

```
AND RANK() OVER (
    PARTITION BY c.id_cliente
    ORDER BY SUM(...) DESC
) <= 3
```

En Oracle una función analítica no puede filtrarse directamente en el WHERE del mismo bloque, porque el WHERE se evalúa antes de que las funciones analíticas produzcan su resultado.

```
Orden lógico simplificado:
FROM / JOIN
WHERE
GROUP BY
HAVING
SELECT / funciones analíticas
ORDER BY
```

Corrección: calcule primero el ranking en una CTE o subconsulta y filtre puesto <= 3 en un SELECT externo.

3.3 Defecto 2 - El LEFT JOIN se convierte en un filtro excluyente

```
LEFT JOIN devolucion dv
       ON dv.id_pedido = p.id_pedido
```

```
WHERE dv.motivo <> 'GARANTIA'
```

Un pedido sin devoluciones produce NULL en las columnas de DEVOLUCION. La condición NULL <> 'GARANTIA' no es TRUE, por lo que la fila desaparece. Esto contradice el requisito de mostrar pedidos sin devoluciones con num_devoluciones = 0.

Corrección: agregar primero las devoluciones válidas y luego unir ese resultado mediante LEFT JOIN; para los pedidos sin coincidencia, usar NVL(..., 0).

3.4 Defecto 3 - Join incompleto hacia DEVOLUCION

La relación definida por el modelo es compuesta: (id_pedido, num_linea) de DEVOLUCION referencia (id_pedido, num_linea) de DETALLE_PEDIDO. Unir únicamente por id_pedido puede asociar una devolución de una línea con otras líneas del mismo pedido.

```
ON  dv.id_pedido = d.id_pedido
AND dv.num_linea = d.num_linea
```

3.5 Defecto 4 - Multiplicación del valor del pedido

Como una misma línea puede tener varias devoluciones, unir DETALLE_PEDIDO con DEVOLUCION antes de sumar puede duplicar o multiplicar el valor vendido. Ejemplo: una línea de $800.000 con dos devoluciones puede aparecer dos veces y terminar sumando $1.600.000.

Nota docente: Regla general: si dos relaciones tienen granularidades distintas, agregue cada una primero y únalas después.

3.6 Defecto adicional - RANK puede devolver más de tres filas

Si existe empate en el tercer valor, RANK asigna el mismo puesto a ambas filas y puesto <= 3 puede devolver más de tres pedidos. Si el requerimiento significa exactamente tres pedidos, use ROW_NUMBER con un criterio determinista de desempate, por ejemplo id_pedido.

3.7 Construcción paso a paso de la solución

Paso 1. Calcular el valor total de cada pedido

```
SELECT p.id_cliente,
       p.id_pedido,
       p.fecha_pedido,
       SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS valor_pedido
FROM pedido p
JOIN detalle_pedido d
  ON d.id_pedido = p.id_pedido
WHERE p.fecha_pedido >= DATE '2025-01-01'
  AND p.fecha_pedido <  DATE '2026-01-01'
GROUP BY p.id_cliente, p.id_pedido, p.fecha_pedido;
```

Paso 2. Contar devoluciones por pedido

```
SELECT id_pedido,
       COUNT(*) AS num_devoluciones
FROM devolucion
WHERE motivo <> 'GARANTIA'
   OR motivo IS NULL
GROUP BY id_pedido;
```

El resultado de esta consulta se unirá después con LEFT JOIN. Los pedidos que no aparezcan aquí recibirán cero mediante NVL.

Paso 3. CTE de valor por pedido

```
WITH valor_pedidos AS
(
    SELECT c.id_cliente,
           c.nombre,
           p.id_pedido,
           p.fecha_pedido,
           SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS valor_pedido
    FROM cliente c
    JOIN pedido p
      ON p.id_cliente = c.id_cliente
    JOIN detalle_pedido d
      ON d.id_pedido = p.id_pedido
    WHERE c.segmento = 'PREMIUM'
      AND p.fecha_pedido >= DATE '2025-01-01'
      AND p.fecha_pedido <  DATE '2026-01-01'
    GROUP BY c.id_cliente, c.nombre, p.id_pedido, p.fecha_pedido
)
```

Paso 4. CTE de devoluciones

```
devoluciones_pedido AS
(
    SELECT id_pedido,
           COUNT(*) AS num_devoluciones
    FROM devolucion
    WHERE motivo <> 'GARANTIA'
       OR motivo IS NULL
    GROUP BY id_pedido
)
```

Paso 5. Acumulado y Top-3

El acumulado se calcula cronológicamente. Como puede haber dos pedidos en la misma fecha, se agrega id_pedido como desempate y se especifica una ventana ROWS.

```
SUM(vp.valor_pedido) OVER (
    PARTITION BY vp.id_cliente
    ORDER BY vp.fecha_pedido, vp.id_pedido
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS acumulado
ROW_NUMBER() OVER (
    PARTITION BY vp.id_cliente
    ORDER BY vp.valor_pedido DESC, vp.id_pedido
) AS puesto
```

3.8 Consulta completa corregida

```
WITH valor_pedidos AS
(
    SELECT c.id_cliente,
           c.nombre,
           p.id_pedido,
           p.fecha_pedido,
           SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS valor_pedido
    FROM cliente c
    JOIN pedido p
      ON p.id_cliente = c.id_cliente
    JOIN detalle_pedido d
      ON d.id_pedido = p.id_pedido
    WHERE c.segmento = 'PREMIUM'
      AND p.fecha_pedido >= DATE '2025-01-01'
      AND p.fecha_pedido <  DATE '2026-01-01'
```

```
    GROUP BY c.id_cliente, c.nombre, p.id_pedido, p.fecha_pedido
),
devoluciones_pedido AS
(
    SELECT id_pedido,
           COUNT(*) AS num_devoluciones
    FROM devolucion
    WHERE motivo <> 'GARANTIA'
       OR motivo IS NULL
    GROUP BY id_pedido
),
datos AS
(
    SELECT vp.id_cliente,
           vp.nombre,
           vp.id_pedido,
           vp.fecha_pedido,
           vp.valor_pedido,
           SUM(vp.valor_pedido) OVER (
               PARTITION BY vp.id_cliente
               ORDER BY vp.fecha_pedido, vp.id_pedido
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
           ) AS acumulado,
           NVL(dp.num_devoluciones, 0) AS num_devoluciones,
           ROW_NUMBER() OVER (
               PARTITION BY vp.id_cliente
               ORDER BY vp.valor_pedido DESC, vp.id_pedido
           ) AS puesto
    FROM valor_pedidos vp
    LEFT JOIN devoluciones_pedido dp
      ON dp.id_pedido = vp.id_pedido
)
SELECT id_cliente,
       nombre,
       id_pedido,
       fecha_pedido,
       valor_pedido,
       acumulado,
       num_devoluciones,
       puesto
FROM datos
WHERE puesto <= 3
ORDER BY id_cliente, puesto;
```

3.9 Pregunta 1.c - RANGE vs ROWS

Para el cliente 100 se tienen los siguientes valores ya calculados:

Pedido	Fecha	Valor
9001	2025-03-04	500.000
9002	2025-03-11	1.200.000
9003	2025-03-11	300.000
9004	2025-04-02	800.000

La consulta original usa SUM(...) OVER (PARTITION BY ... ORDER BY fecha_pedido) sin especificar ROWS. Con empates en la fecha, Oracle trata las filas con la misma clave de ordenamiento como pares dentro de la ventana lógica.

Pedido	Acumulado original	Acumulado correcto
9001	500.000	500.000
9002	2.000.000	1.700.000
9003	2.000.000	2.000.000

9004	2.800.000	2.800.000

```
    ORDER BY fecha_pedido, id_pedido
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

4. Pregunta 2 - Construcción de un informe gerencial

El informe se agrupa por región, mes y categoría raíz. La venta neta es venta menos devolución, pero cada devolución debe imputarse al mes de fecha_devolucion. Además, se requiere variación mensual, venta anual y Top-3 de categorías por región.

4.1 Estrategia de solución

```
1. Resolver categoría -> categoría raíz
2. Calcular ventas por mes
3. Calcular devoluciones por mes
4. Unir ventas y devoluciones
5. Calcular venta neta
6. Aplicar LAG para variación mensual
7. Calcular venta anual
8. Aplicar DENSE_RANK por región
9. Filtrar Top-3
```

4.2 CTE 1 - Resolver la categoría raíz con CONNECT BY

```
SELECT id_categoria,
       CONNECT_BY_ROOT id_categoria AS id_categoria_raiz,
       CONNECT_BY_ROOT nombre AS categoria_raiz
FROM categoria
START WITH id_categoria_padre IS NULL
CONNECT BY NOCYCLE
       PRIOR id_categoria = id_categoria_padre;
```

START WITH identifica las raíces. PRIOR define la relación padre-hijo. CONNECT_BY_ROOT conserva para cada descendiente la categoría de primer nivel. NOCYCLE evita que una relación cíclica rompa el recorrido.

4.3 CTE 2 - Ventas mensuales

```
ventas_mes AS
(
    SELECT s.region,
           TRUNC(p.fecha_pedido, 'MM') AS mes,
           cr.id_categoria_raiz,
           cr.categoria_raiz,
           SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS valor_vendido
    FROM pedido p
    JOIN sucursal s
      ON s.id_sucursal = p.id_sucursal
    JOIN detalle_pedido d
      ON d.id_pedido = p.id_pedido
    JOIN producto pr
      ON pr.id_producto = d.id_producto
    JOIN categoria_raiz cr
      ON cr.id_categoria = pr.id_categoria
    WHERE p.estado = 'FACTURADO'
      AND p.fecha_pedido >= DATE '2025-01-01'
      AND p.fecha_pedido <  DATE '2026-01-01'
    GROUP BY s.region,
             TRUNC(p.fecha_pedido, 'MM'),
             cr.id_categoria_raiz,
             cr.categoria_raiz
)
```

4.4 CTE 3 - Devoluciones mensuales

La devolución se imputa al mes de fecha_devolucion, no al mes del pedido. El precio y el descuento se toman de la línea original.

```
devoluciones_mes AS
(
    SELECT s.region,
           TRUNC(dv.fecha_devolucion, 'MM') AS mes,
           cr.id_categoria_raiz,
           cr.categoria_raiz,
           SUM(
               dv.cantidad_devuelta *
               d.precio_unitario *
               (1 - d.descuento)
           ) AS valor_devuelto
    FROM devolucion dv
    JOIN detalle_pedido d
      ON d.id_pedido = dv.id_pedido
     AND d.num_linea = dv.num_linea
    JOIN pedido p
      ON p.id_pedido = d.id_pedido
    JOIN sucursal s
      ON s.id_sucursal = p.id_sucursal
    JOIN producto pr
      ON pr.id_producto = d.id_producto
    JOIN categoria_raiz cr
      ON cr.id_categoria = pr.id_categoria
    WHERE p.estado = 'FACTURADO'
      AND dv.fecha_devolucion >= DATE '2025-01-01'
      AND dv.fecha_devolucion <  DATE '2026-01-01'
    GROUP BY s.region,
             TRUNC(dv.fecha_devolucion, 'MM'),
             cr.id_categoria_raiz,
             cr.categoria_raiz
)
```

4.5 CTE 4 - FULL OUTER JOIN y venta neta

El FULL OUTER JOIN se justifica porque puede existir un mes con ventas pero sin devoluciones, o un mes con devoluciones de pedidos anteriores pero sin ventas de esa categoría.

```
SELECT COALESCE(v.region, d.region) AS region,
       COALESCE(v.mes, d.mes) AS mes,
       COALESCE(v.id_categoria_raiz, d.id_categoria_raiz) AS id_categoria_raiz,
       COALESCE(v.categoria_raiz, d.categoria_raiz) AS categoria_raiz,
       NVL(v.valor_vendido, 0) - NVL(d.valor_devuelto, 0) AS venta_neta
FROM ventas_mes v
FULL OUTER JOIN devoluciones_mes d
  ON  d.region = v.region
  AND d.mes = v.mes
  AND d.id_categoria_raiz = v.id_categoria_raiz
```

4.6 CTE 5 - LAG y venta anual

```
LAG(venta_neta) OVER (
    PARTITION BY region, id_categoria_raiz
    ORDER BY mes
) AS venta_anterior

SUM(venta_neta) OVER (
    PARTITION BY region, id_categoria_raiz
) AS venta_anual
```

La variación porcentual puede calcularse con (actual - anterior) / anterior × 100. NULLIF(venta_anterior, 0) evita división por cero.

4.7 CTE 6 - Ranking anual por región

El enunciado dice que los empates comparten puesto y no dejan huecos. Esa semántica corresponde a DENSE_RANK.

```
DENSE_RANK() OVER (
    PARTITION BY region
    ORDER BY venta_anual DESC
) AS puesto
```

4.8 Consulta completa

```
WITH categoria_raiz AS
(
    SELECT id_categoria,
           CONNECT_BY_ROOT id_categoria AS id_categoria_raiz,
           CONNECT_BY_ROOT nombre       AS categoria_raiz
    FROM categoria
    START WITH id_categoria_padre IS NULL
    CONNECT BY NOCYCLE PRIOR id_categoria = id_categoria_padre
),
ventas_mes AS
(
    SELECT s.region,
           TRUNC(p.fecha_pedido, 'MM') AS mes,
           cr.id_categoria_raiz,
           cr.categoria_raiz,
           SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS valor_vendido
    FROM pedido p
    JOIN sucursal s ON s.id_sucursal = p.id_sucursal
    JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
    JOIN producto pr ON pr.id_producto = d.id_producto
    JOIN categoria_raiz cr ON cr.id_categoria = pr.id_categoria
    WHERE p.estado = 'FACTURADO'
      AND p.fecha_pedido >= DATE '2025-01-01'
      AND p.fecha_pedido <  DATE '2026-01-01'
    GROUP BY s.region, TRUNC(p.fecha_pedido, 'MM'),
             cr.id_categoria_raiz, cr.categoria_raiz
),
devoluciones_mes AS
(
    SELECT s.region,
           TRUNC(dv.fecha_devolucion, 'MM') AS mes,
           cr.id_categoria_raiz,
           cr.categoria_raiz,
           SUM(dv.cantidad_devuelta * d.precio_unitario * (1 - d.descuento)) AS valor_devuelto
    FROM devolucion dv
    JOIN detalle_pedido d
      ON d.id_pedido = dv.id_pedido
     AND d.num_linea = dv.num_linea
    JOIN pedido p ON p.id_pedido = d.id_pedido
    JOIN sucursal s ON s.id_sucursal = p.id_sucursal
    JOIN producto pr ON pr.id_producto = d.id_producto
    JOIN categoria_raiz cr ON cr.id_categoria = pr.id_categoria
    WHERE p.estado = 'FACTURADO'
      AND dv.fecha_devolucion >= DATE '2025-01-01'
      AND dv.fecha_devolucion <  DATE '2026-01-01'
    GROUP BY s.region, TRUNC(dv.fecha_devolucion, 'MM'),
             cr.id_categoria_raiz, cr.categoria_raiz
),
mensual AS
(
```

```
    SELECT COALESCE(v.region, d.region) AS region,
           COALESCE(v.mes, d.mes) AS mes,
           COALESCE(v.id_categoria_raiz, d.id_categoria_raiz) AS id_categoria_raiz,
           COALESCE(v.categoria_raiz, d.categoria_raiz) AS categoria_raiz,
           NVL(v.valor_vendido, 0) - NVL(d.valor_devuelto, 0) AS venta_neta
    FROM ventas_mes v
    FULL OUTER JOIN devoluciones_mes d
      ON  d.region = v.region
      AND d.mes = v.mes
      AND d.id_categoria_raiz = v.id_categoria_raiz
),
analiticas AS
(
    SELECT m.*,
           LAG(venta_neta) OVER (
               PARTITION BY region, id_categoria_raiz
               ORDER BY mes
           ) AS venta_anterior,
           SUM(venta_neta) OVER (
               PARTITION BY region, id_categoria_raiz
           ) AS venta_anual
    FROM mensual m
),
ranking AS
(
    SELECT a.*,
           DENSE_RANK() OVER (
               PARTITION BY region
               ORDER BY venta_anual DESC
           ) AS puesto
    FROM analiticas a
)
SELECT region,
       mes,
       categoria_raiz,
       venta_neta,
       ROUND(
           100 * (venta_neta - venta_anterior)
           / NULLIF(venta_anterior, 0),
           2
       ) AS variacion_pct,
       venta_anual,
       puesto
FROM ranking
WHERE puesto <= 3
ORDER BY region, puesto, categoria_raiz, mes;
```

4.9 Pregunta 2.b - Por qué no usar self-joins fijos

Un número fijo de self-joins solo soporta una profundidad conocida. Si hoy la jerarquía tiene cuatro niveles y mañana se agrega un quinto, una solución con self-joins debe modificarse. Una consulta jerárquica con CONNECT BY recorre cualquier profundidad sin cambiar el SQL.

- START WITH id_categoria_padre IS NULL: define las categorías raíz.
- PRIOR id_categoria = id_categoria_padre: define la relación padre → hijo.
- CONNECT BY NOCYCLE: evita ciclos infinitos si existe una relación cíclica por error.

4.10 Pregunta 2.c - Por qué ROWNUM <= 3 no resuelve el Top-N por grupo

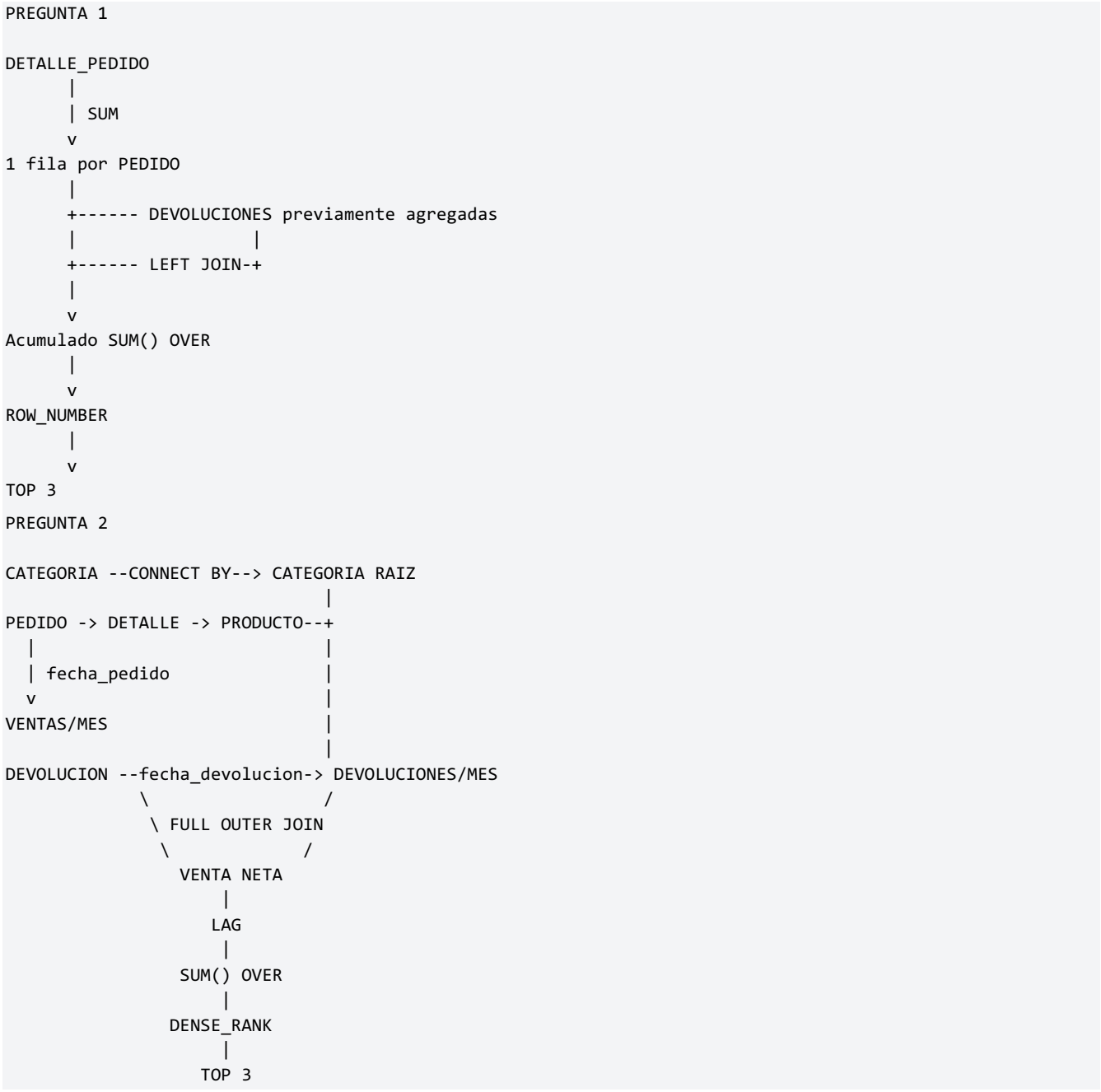
ROWNUM se asigna antes del ORDER BY del mismo bloque. Por ello, ROWNUM <= 3 toma las primeras tres filas que llegan y después ORDER BY ordena únicamente esas filas. Además, es un Top-3 global, no un Top-3 por región.

Alternativas correctas:

- Para Top-N por grupo: ROW_NUMBER, RANK o DENSE_RANK con PARTITION BY y filtro externo.
- Para Top-N global: ordenar en un bloque adecuado y usar FETCH FIRST n ROWS ONLY.

Nota docente: En este ejercicio, DENSE_RANK es la función correcta porque el enunciado exige que los empates compartan puesto sin dejar huecos.

5. Resumen visual para el tablero



6. Ideas clave que deben llevarse los estudiantes

- La granularidad debe definirse antes de escribir joins complejos.
- Un LEFT JOIN puede dejar de conservar filas si se filtran columnas de la tabla opcional en WHERE.
- Agregar antes de unir evita multiplicaciones de filas.
- Las funciones analíticas se calculan después de WHERE/GROUP BY y se filtran en un nivel externo.
- ROWS y RANGE no son equivalentes cuando existen empates en la columna de ordenamiento.
- CONNECT BY permite resolver jerarquías de profundidad variable.
- LAG compara una fila con la anterior dentro de una partición.
- DENSE_RANK es apropiado cuando los empates comparten puesto y no dejan huecos.
- ROWNUM no reemplaza un Top-N por grupo.